CP255 - W8_S1 Bar Chart Best Practices Answer Key

Your Turn:
Dataset: Number of potholes reported per month in a city.
Steps:

- **Run the Before cell** — notice the distracting multi-color palette
- **Fix it:** use a single appropriate color for all bars
- **Bonus:** Add a subtitle explaining what the data is (e.g., ax.set_title('...\nsubtitle here'))

Answer:

Python

```
# YOUR TURN — use a single color

fig, ax = plt.subplots(figsize=(9, 4))

# --- Add your code here ---
# Replace the rainbow color palette with one consistent color
ax.bar(
  potholes['Month'],
  potholes['Reports'],
  color= '#ffda99'
  )

ax.set_ylabel('Pothole Reports')
ax.set_title('CORRECT Plot: Monthly Pothole Reports')
# find the code that sets the title and change it to show this is the correct plot

ax.spines[['top','right']].set_visible(False)
plt.tight_layout()
plt.show()
```

Your Turn:
Dataset: Park acreage across city departments.
Steps:

- Run the Before cell
- Fix the color issue: use a single muted color for all bars

Answer:

```python
# YOUR TURN — fix the colors

fig, ax = plt.subplots(figsize=(6, 4))

# --- Add your code here ---

ax.barh(
  parks['Department'],
  parks['Acres'],
  color= '#91F527'
  )

ax.set_xlabel('Acres of Parkland')
ax.set_title('Parkland Managed by City Department')

ax.spines[['top','right']].set_visible(False)

plt.tight_layout()
plt.show()
```

Your Turn:
Dataset: Average wait time (minutes) at permit offices across different city departments.
Steps:

- **Run the Before cell** — notice how the rotated labels look
- Convert to a horizontal bar chart
- Remember to sort the data before plotting
- **Bonus:** Add a vertical dashed reference line at the city average wait time
- **Hint:** ax.axvline(x=value, linestyle='--', color='gray')

Answer:

```python
# YOUR TURN — convert to horizontal bar chart

fig, ax = plt.subplots(figsize=(6, 4))
```

```
# Step 1: Sort the data
wait_sorted = wait_times.sort_values('Avg_Wait_Min')

# Step 2: plot horizontally
# --- Add your code here ---

ax.barh(
  wait_sorted['Department'],
  wait_sorted['Avg_Wait_Min'],
  color='#a29bfe'
  )

ax.set_xlabel('Avg Wait Time (minutes)')
ax.set_title('Average Permit Office Wait Time by Department')
ax.spines[['top','right']].set_visible(False)
plt.tight_layout()
plt.show()
```

Your Turn:
Dataset: Number of 311 service requests by request type.
Steps:

- **Run the Before cell**
- Convert to a horizontal bar chart and sort
- **Bonus:** Combine with Activity 3 — use a single color and highlight the top category

Answer:

```
Python
# YOUR TURN — horizontal bar chart + sort

fig, ax = plt.subplots(figsize=(6, 4))

# --- Add your code here ---

requests_311 = requests_311.sort_values("Count")
```

```
ax.barh(
  requests_311['Request_Type'],
  requests_311['Count'],
  color='#fd79a8'
  )

ax.set_xlabel('Number of Requests')
ax.set_title('311 Service Requests by Type')
ax.spines[['top','right']].set_visible(False)
plt.tight_layout()
plt.show()
```

Your Turn:
Dataset: Number of bike lane miles added per year in a city's five districts.
Steps:

- **Run the "Before" cell**
- In the empty cell, highlight the district with the most and fewest miles added
- Add text labels only to those two highlighted bars
- **Bonus:** Try a different highlight color — pick one from colorbrewer2.org

Answer:

Python
```
# YOUR TURN — Practice A
# Highlight the district with the most AND fewest miles added, label both bars

# Step 1: find the max and min values, replace ___ with your code
max_val = bike_lanes['Miles_Added'].max()  # hint: what method gives you the largest value?
min_val = bike_lanes['Miles_Added'].min()  # hint: opposite of the above

# Step 2: build the colors array
# Fill in the blanks — the structure is given to you, you just need the colors
colors = np.where(
  bike_lanes['Miles_Added'] == max_val, '#3B85B8',  # pick a dark highlight color
  np.where(
```

```
    bike_lanes['Miles_Added'] == min_val, '#3B85B8',  # same highlight color
    '#B0D3EB'                          # a muted version of the same color
  )
)

# Step 3: plot — only one word changes from the BEFORE code
fig, ax = plt.subplots(figsize=(6, 3))
ax.barh(bike_lanes['District'], bike_lanes['Miles_Added'], color=colors)  # hint: not a single string anymore

# Step 4: label only the highlighted bars
# .isin() filters to rows where the value matches either max or min
for row in bike_lanes[bike_lanes['Miles_Added'].isin([max_val, min_val])].itertuples():
  ax.text(
    row.Miles_Added + 0.5,  # hint: small number, just enough to clear the bar end
    row.District,
    f'{row.Miles_Added}',
    va='center',
    fontweight='bold'
  )

# Step 5: extend xlim so labels don't get cut off
ax.set_xlim(0, bike_lanes['Miles_Added'].max() * 1.5)  # hint: something slightly bigger than 1.0

ax.set_xlabel('Bike Lane Miles Added')
ax.set_title('New Bike Lane Infrastructure by District')
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.show()
```

Your Turn:
Dataset: Tree canopy coverage (%) across neighborhoods.
Steps:

- **Run the Before cell**
- Highlight the top two neighborhoods (not just the max) in a green color
- Highlight the bottom one neighborhood in red
- Add data labels to all highlighted bars

- **Hint:** Use .nlargest(2) and .nsmallest(1) to find the rows, then use isin() to build your color mask

Answer:

Python

```
# YOUR TURN — Practice B (no hints this time!)
# Dataset: tree canopy coverage across neighborhoods
# Highlight the TOP TWO neighborhoods and the BOTTOM ONE
# Hint: use .nlargest(2) and .nsmallest(1) to find the rows, then .isin() to build your color mask

canopy = pd.DataFrame({
  'Neighborhood': ['Elmwood', 'Rockridge', 'Temescal', 'Fruitvale',
           'Montclair', 'Dimond', 'Laurel'],
  'Canopy_Pct': [31, 45, 18, 12, 52, 27, 33]
}).sort_values('Canopy_Pct')

max_val = canopy['Canopy_Pct'].max()
second_largest_val = canopy['Canopy_Pct'].nlargest(2).iloc[-1]
min_val = canopy['Canopy_Pct'].min()

top_two = canopy['Canopy_Pct'].nlargest(2)
bottom_one = canopy['Canopy_Pct'].nsmallest(1)

colors = np.where(
  canopy['Canopy_Pct'].isin(top_two),
  '#2ecc71',    # green for top two
  np.where(
    canopy['Canopy_Pct'].isin(bottom_one),
    '#e74c3c',  # red for bottom one
    '#d5f5e3'   # muted color for others
  )
)

fig, ax = plt.subplots(figsize=(7, 4))

ax.barh(
  canopy['Neighborhood'],
  canopy['Canopy_Pct'],
  color=colors
)
```

```
for row in canopy[
  canopy['Canopy_Pct'].isin(
    [max_val, second_largest_val, min_val]
  )
].itertuples():
  ax.text(
    row.Canopy_Pct + 0.5,
    row.Neighborhood,
    f'{row.Canopy_Pct}',
    va='center',
    fontweight='bold'
  )

ax.set_xlim(0, canopy['Canopy_Pct'].max() * 1.5)
ax.set_xlabel('Tree Canopy Coverage (%)')
ax.set_title('Tree Canopy Coverage by Neighborhood')
ax.spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.show()
```

Your Turn:
Dataset: Green space per capita (sq ft) across city council districts.
Steps:

- **Run the Before cell**
- Highlight District 7 (the one with the least green space)
- Mute all other bars
- **Add an annotation:** "District 7 residents have the least access to green space."

Answer:

Python

```
# YOUR TURN — highlight District 7, mute the rest, add annotation

FOCUS = 'District 7'
```

```
fig, ax = plt.subplots(figsize=(7, 5))

# Step 1: build colors with np.where
colors = np.where(green_space['District'] == FOCUS, '#e67e22', '#f5cba7')


# Step 2: plot
ax.barh(green_space['District'], green_space['Sq_Ft_Per_Capita'], color=colors)

# Step 3: add annotation
# --- ax.annotate(...) here ---
focus_val = green_space.loc[green_space['District'] == FOCUS, 'Sq_Ft_Per_Capita'].values[0]

ax.annotate(
  'District 7 residents have the least access to green space.',
  xy=(focus_val, FOCUS),       # arrow TIP: points at the bar end
  xytext=(focus_val + 20, FOCUS), # arrow TAIL: where the text sits
  fontsize=9,
  color='#7f8c8d',
  va='center',
  arrowprops=dict(arrowstyle='->', color='#7f8c8d', lw=1.2)
)

ax.set_xlabel('Green Space per Capita (sq ft)')
ax.set_title('Green Space per Capita by Council District')
ax.spines[['top','right']].set_visible(False)
plt.tight_layout()
plt.show()
```

Your Turn:
Dataset: Pedestrian injury rate (per 10k residents) by neighborhood.
Steps:

- **Run the Before cell**
- Highlight the highest injury rate neighborhood
- Mute all others
- Write your own annotation explaining why this neighborhood deserves attention
- **Bonus:** Also add a data label showing the exact value for the highlighted bar

Answer:

```python
# YOUR TURN — highlight the highest injury rate, mute the rest

fig, ax = plt.subplots(figsize=(6, 3))

FOCUS = ped_safety.loc[
  ped_safety['Injury_Rate'].idxmax(),
  'Neighborhood'
]

colors = np.where(
  ped_safety['Neighborhood'] == FOCUS,
  '#e74c3c',
  '#f5cba7'
)

ax.barh(
  ped_safety['Neighborhood'],
  ped_safety['Injury_Rate'],
  color=colors
)

focus_val = ped_safety.loc[
  ped_safety['Neighborhood'] == FOCUS,
  'Injury_Rate'
].values[0]

ax.annotate(
  'This neighborhood has the highest pedestrian injury rate.',
  xy=(focus_val, FOCUS),
  xytext=(focus_val + 2, FOCUS),
  fontsize=9,
  color='#7f8c8d',
  va='center',
  arrowprops=dict(
    arrowstyle='->',
    color='#7f8c8d',
    lw=1.2
  )
```

```
)

ax.text(
  focus_val + 0.2,
  FOCUS,
  f'{focus_val}',
  va='center',
  fontweight='bold'
)

ax.set_xlabel('Pedestrian Injuries per 10,000 Residents')
ax.set_title('Pedestrian Injury Rate by Neighborhood')
ax.spines[['top','right']].set_visible(False)

plt.tight_layout()
plt.show()
```

Your Turn: Convert the bar chart below into a lollipop plot.
Dataset: Average commute time (minutes) by transit mode in a mid-sized US city.
Steps:

- **Run the "Before" cell to see the bar chart**
- In the empty cell below it, recreate it as a lollipop plot
- **Bonus:** Change the dot color to #2ecc71 (green)

Python

```
# YOUR TURN — convert to a lollipop plot
# Use the same 'commute' DataFrame above
# Hint: you need ax.hlines() and ax.plot()
# Create a dataframe
commute = pd.DataFrame({
  'Mode': ['Bus', 'Light Rail', 'Subway', 'Ferry', 'Bike Share', 'Walking'],
  'Avg_Commute_Min': [38, 29, 24, 45, 18, 12]
})

commute = commute.sort_values('Avg_Commute_Min')

fig, ax = plt.subplots(figsize=(7, 4))
```

```
# Lollipop stems
ax.hlines(
  y=commute['Mode'],
  xmin=0,
  xmax=commute['Avg_Commute_Min'],
  color='steelblue',
  linewidth=2.5
)

# Lollipop heads
ax.plot(
  commute['Avg_Commute_Min'],
  commute['Mode'],
  'o',
  color='#2ecc71',
  markersize=10
)

ax.set_xlabel('Average Commute Time (minutes)')
ax.set_title('Average Commute Time by Transit Mode')

plt.tight_layout()
plt.show()
```